Machine Learning-Assisted Priority Scheduling and Reinforcement Learning-Based Adaptive Round Robin: A Pilot Simulation Study with a Correlation-Guided Burst Predictor

Aaditya Kumar, Mohnish Tulsian, Achintya B S
Department of Computer Science and Engineering – Artificial Intelligence and Machine Learning
PES University, Bangalore, India

Abstract

Classical CPU scheduling policies such as Shortest-Remaining-Time-First and priority scheduling are near-optimal when the length of a process's next CPU burst is known in advance, yet this length is never observable in real systems. We present a discrete-event simulation study comparing six scheduling algorithms built around this information gap: non-preemptive Priority Scheduling (A); a Weighted ML variant (B) that blends a regression-predicted burst length into the priority score; a Protected ML variant (C) that adds explicit anti-starvation safeguards to B; fixed-quantum Round Robin (D); a rule-based adaptive Round Robin (E); and a Round Robin variant whose quantum is chosen by a tabular Q-learning agent (F). We also propose and evaluate a Correlation-Weighted Adaptive Burst Predictor (CWABP) for Shortest-Remaining-Time-First scheduling against classical exponential averaging. All six algorithms, five candidate burst-prediction models, the Q-learning agent, and CWABP are fully implemented in an open-source simulator (`schedsim`); every number reported here is genuine output from that code, not a placeholder. On a reduced-scale pilot run (25 processes per workload, 65 paired test workloads), Weighted ML Priority Scheduling reduces mean waiting time by 32% relative to traditional Priority Scheduling ($p < 10^{-9}$, Wilcoxon signed-rank, Holm-corrected), while the Protected variant trades back part of that gain for a 27% reduction in worst-case waiting time. Neither the rule-based nor the Q-learning adaptive Round Robin policy outperformed fixed-quantum Round Robin at this pilot scale, a negative result we report plainly together with the scale limitations likely responsible for it. CWABP matched or modestly improved on exponential averaging across seven synthetic I/O workload types, with the largest gains (-5.7% waiting time, -25% RMSE) on a feature-driven, phase-changing workload. We report these as *pilot-scale* findings and specify the exact configuration needed to reproduce or scale up every result.

Index Terms—CPU scheduling, Priority Scheduling, Round Robin, time quantum, burst prediction, exponential averaging, Shortest Remaining Time First, reinforcement learning, Q-learning, fairness, starvation.

I. INTRODUCTION

A CPU scheduler must repeatedly decide which ready process runs next. Classical policies such as Shortest Job First and Shortest-Remaining-Time-First are provably strong *if* the scheduler knows each process's remaining burst length in advance; real schedulers never have this information and instead rely on static priorities, fixed time slices, or simple estimators [1]. This paper investigates, through simulation rather than kernel modification, whether (i) a machine-learning estimate of burst length can improve priority-based scheduling, and (ii) a Round Robin time quantum chosen adaptively – by hand-written rules or by a reinforcement learning (RL) agent – outperforms a fixed quantum. A 2017 survey [2] motivates this question; Section II treats it as a pointer into the literature, re-checking its central claims against primary sources rather than repeating them uncritically.

This paper makes no claim that ML or RL improves scheduling in general. It tests whether, and under what pilot-scale conditions, these methods help, treating fairness loss, tail-latency cost, and context-switch overhead as first-class outcomes. All code is openly available; every table in this paper is generated directly from that code's output (see §VI).

Contributions. (1) A single open-source simulator evaluating all six algorithms under identical workloads and counting conventions; (2) a direct, statistically tested comparison between performance-oriented and fairness-protected ML priority ranking; (3) separate accounting of burst-prediction accuracy versus scheduling outcome; (4) a frozen evaluation protocol for CWABP against exponential averaging, run end-to-end; and (5) full transparency about pilot-scale limitations, including two implementation bugs this project's own test suite caught during development (§VI-C).

II. RELATED WORK

We re-verified the central claims of a 2017 survey [2] against original sources. Negi and Kishore [3] modified a Linux 2.4 kernel scheduler and report a 1.4–5.8% turnaround-time reduction via decision-tree-learned time-slice classes, attributed to fewer context switches – confirmed at the abstract level. Helmy et al. [4] report strong grid-burst prediction (k-NN best) on the GWA-T-4 trace; the specific correlation/RAE figures quoted by the survey were not independently re-verified. Two sources cited only through the survey (a 2004 Negi & Kishore paper and a Dolev/Mendelson/Shilman technical report) could not be located at all.

More recent dynamic-quantum Round Robin work includes DABRR [5], an effective-time-slice RR [6], k-means-based RR [7], and an agent-based adaptive RR compared

against twelve baselines [8]. RL has been applied to real scheduler parameters by Han and Lee [9] (Android's Energy-Aware Scheduler) and, in preprint form, to general adaptive resource scheduling [10]. Within a necessarily limited search, we did not find a peer-reviewed study comparing fixed, rule-based, and Q-learning Round Robin quantum selection *together* under shared tie-breaking rules while reporting worst-case waiting and starvation counts, nor a protected-vs-unprotected ML priority comparison reporting explicit inversion and starvation statistics – the gap this study targets.

Table 1: Positioning relative to closely related work.

Work	Technique	Evaluated against
[3]	Decision-tree time-slice classes, real kernel	Turnaround time only
[4]	SVM/k-NN/ANN/DT burst estimation	Grid traces, no scheduling sim
[5, 6]	Rule-based dynamic RR quantum	AWT/ATAT, single RR variant
[7]	k-means-based RR quantum	Not fully verified
[8]	Agent-based rules, 12-way comparison	No worst-case/starvation metrics
[9]	Policy-gradient RL, real scheduler	Android power/benchmarks
[10]	Q-learning, general resources	Preprint, unreviewed
This work	**5 ML models + rules + tabular Q-learning, 6-way**	**Worst-case, starvation, fairness, stat. tests**

III. SYSTEM DESIGN

3.1 Simulator Architecture

`schedsim` runs five stages in strict sequence: (1) seeded workload generation, (2) burst-length prediction fitted on training data and selected on validation data, (3) discrete-event scheduling restricted to information a real scheduler could observe, (4) metrics computation from the resulting event log, and (5) statistical analysis. Round Robin quantum policies (E, F) never receive true burst lengths or future arrivals – only queue length, accumulated waiting time, and *predicted* remaining bursts.

3.2 Synthetic Workloads

Each workload is a deterministic function of a specification and an integer seed; training, validation, and test splits use disjoint seed ranges by construction. Processes carry four synthetic attributes (named after, but not drawn from, real Linux process fields) whose correlation with true log-burst length is a controllable design parameter, ranging from strong ($\rho = 0.95$) to deliberately misleading ($\rho = -0.70$). Ten in-distribution workload categories (varying burst-length distribution, arrival pattern, priority assignment, and feature strength) appear in all three splits; three *shifted* categories – misleading features, heavy-tailed bursts, and a 120-process queue – appear only in the test split, to probe generalisation.

IV. ALGORITHMS AND MACHINE LEARNING MODELS

4.1 Common Definitions

For process i with arrival a_i, first dispatch s_i, completion c_i, and true burst b_i: turnaround $T_i = c_i - a_i$, waiting $W_i = T_i - b_i$, response $R_i = s_i - a_i$. A context switch is charged (1.0 time unit in the main experiments) whenever dispatch switches between two different processes with no intervening idle interval.

4.2 The Five Candidate Burst Prediction Models

A real operating system cannot observe future execution requirements. To bridge this information gap, candidate regression estimators predict the expected CPU burst length $\hat{B}_i$ based strictly on four observable memory management attributes: virtual memory size (`vsize`), code segment size (`text`), end of code boundary (`endcode`), and dynamic library allocation size (`vmlib`). True burst length, arrival time, and static priority are structurally omitted to prevent target leakage. We evaluate five distinct paradigms:

1. **Ordinary Least Squares Linear Regression (`linear`):** Fits a linear hyperplane $\hat{y} = \mathbf{w}^T\mathbf{x} + b$ minimising residual sum of squares. Because inference requires only $\mathcal{O}(d)$ floating point operations, it imposes negligible dispatch overhead inside an OS scheduler.
2. **k-Nearest Neighbors Regressor (`knn`):** Instance-based non-parametric regression with $k = 5$ neighbors. Predictions represent the uniform average of the k closest historical processes in standardized Euclidean feature space. While effective in small-scale trace studies, neighbor search incurs $\mathcal{O}(N \cdot d)$ complexity per dispatch.
3. **Classification and Regression Trees (`tree`):** A CART decision tree constrained to $\mathrm{max_depth} = 6$. It recursively partitions the feature space along orthogonal hyperplanes minimising mean squared error, yielding explicit if-else rules readily implementable in kernel C code.
4. **Random Forest Regressor (`random_forest`):** An ensemble of 100 decorrelated decision trees ($\mathrm{max_depth} = 8$) trained on bootstrap subsets of the training workloads. Ensembling mitigates high-variance outlier predictions at the expense of $100\times$ higher inference latency.
5. **Support Vector Regressor (`svr`):** Radial basis function (RBF) kernel SVR ($C = 10.0, \epsilon = 0.5$). It maps features into an infinite-dimensional RKHS, fitting a flat margin tube that ignores deviations under ϵ.

4.3 Priority Family (A, B, C)

A dispatches the ready process with the smallest priority number (FCFS/ID tie-break), running to completion. **B** blends min-max normalised priority $\hat{P}_i$ and predicted burst $\hat{B}_i$ into $\mathrm{Score}_i = \alpha\hat{P}_i + (1-\alpha)\hat{B}_i$, dispatching the lowest score; α=1 reproduces A exactly (verified by unit test; §VI-B). **C** adds two safeguards on top of B's score: a *maximum-wait override* that, once any ready process has waited $\geq$

$W_{\max}$, dispatches the earliest-arrived overdue process regardless of score; and *bounded inversion*, restricting dispatch to processes within priority gap G of the best ready priority otherwise. Critically, **the override does not guarantee any process waits at most** $W_{\max}$ – an overdue process must still wait for the currently running process and any earlier-arrived overdue peers – so starvation-threshold violations (wait > 100) are measured, not assumed away. $(W_{\max}, G)$ are selected on validation data to minimise mean max-waiting subject to mean average waiting $\leq 1.10\times$ Algorithm B's; when no grid point satisfies this (observed at this pilot's tuned α=0, where B is already near-SJF-optimal), the feasible-closest point is used instead of an unvalidated default – a concrete answer to a question the original protocol specification left open.

Algorithm 1: Algorithm C: Protected ML Priority Scheduling

Input: Ready set $R(t)$; priorities p_i; predicted bursts $\tilde{b}_i$; waits w_i; threshold $W_{\max}$; gap G
Output: Process $i^\star$ to dispatch at time t

1 $O \leftarrow \{\, i \in R(t) : w_i \geq W_{\max} \,\}$;
2 **if** $O \neq \emptyset$ **then**
3 $\quad i^\star \leftarrow \arg\min_{i \in O}(\text{arrival}_i,\ i)$; `// max-wait override`
4 **else**
5 $\quad p^\star \leftarrow \min_{i \in R(t)} p_i$;
6 $\quad E \leftarrow \{\, i \in R(t) : p_i - p^\star \leq G \,\}$; `// bounded inversion`
7 $\quad i^\star \leftarrow \arg\min_{i \in E} \text{Score}_i$; `// Score as in Alg. B`
8 **return** $i^\star$;

Algorithm 1 summarises the dispatch rule; note that the override branch (lines 2–3) is checked *before*, and takes precedence over, the bounded-inversion branch, which is precisely why it can cross the priority-gap boundary G when a process is overdue.

4.4 Round Robin Family (D, E, F)

D is FIFO Round Robin with constant quantum q, selected on validation data from candidate set $\mathcal{Q} = \{2, 4, 8, 16\}$. **E** chooses among the same four quanta using hand-written thresholds on queue length, median predicted remaining burst, and the dispatched process's accumulated wait – shrinking the quantum under heavy predicted load, growing it for a long-waiting process. **F** replaces E's hand-written rule with a tabular Q-learning agent (Watkins' update, §IV-E) trained offline and evaluated with a frozen, exploration-off policy, so that by evaluation time F behaves as a fixed policy exactly like D and E – just one *produced* by training rather than hand-specified.

4.5 Q-Learning Formulation (Algorithm F)

To dynamically match the time quantum to instantaneous workload characteristics, Algorithm F models quantum selection as an episodic Markov Decision Process:

- **State Space** $\mathcal{S}$ **(**$|\mathcal{S}| = 144$**):** Defined by a Cartesian combination of four binned continuous observations: (1) ready queue length $|R(t)| \in \{[1,4), [4,9), [9,\infty)\}$; (2) mean predicted remaining burst $\bar{b}_{\text{rem}} \in \{[0,4), [4,12), [12,30), [30,\infty)\}$; (3) maximum queued waiting time $W_{\text{queued}} \in \{[0,25), [25,100), [100,\infty)\}$; and (4) load imbalance metric $I = \max(b)/\bar{b} \in \{[0,1.5), [1.5,3.0), [3.0,\infty)\}$. This yields $4 \times 4 \times 3 \times 3 = 144$ discrete states.
- **Action Space** $\mathcal{A}$**:** Index $a \in \{0,1,2,3\}$ choosing $q \in \mathcal{Q} = \{2,4,8,16\}$ time units.
- **Reward Function** $r(s,a)$**:** Penalizes the marginal increase in total queue waiting time between consecutive dispatch events, augmented with regularization terms for response latency and tail starvation:

$$r = -\Delta W_{\text{total}} - 0.1\Delta R_{\text{mean}} - 0.2\Delta W_{\text{tail}} \tag{1}$$

- **Update Rule:** Watkins' tabular Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \eta \left[r + \gamma \max_{a' \in \mathcal{A}} Q(s',a') - Q(s,a) \right] \tag{2}$$

with learning rate $\eta = 0.15$ and discount factor $\gamma = 0.90$. Training applies an ε-greedy exploration schedule decaying from $\varepsilon_0 = 1.0$ to $\varepsilon_{\text{end}} = 0.05$. At evaluation, policy $\pi(s) = \arg\max_a Q(s,a)$ is strictly frozen with ties broken in favor of the smaller quantum.

V. THE CWABP EXTENSION

Exponential averaging, $\hat{\tau}_n = \beta\tau_n + (1-\beta)\hat{\tau}_{n-1}$, is the textbook SRTF/SJF burst estimator ($\beta = 0.5$) but adapts only slowly after a behaviour change and ignores any signal observable *before* a burst begins. We propose the Correlation-Weighted Adaptive Burst Predictor (CWABP): given an observable per-burst scalar x (e.g. preceding I/O size), maintain a sliding window of the last $K = 8$ pairs $(x_j, \ln \tau_j)$ per process. If the sample correlation r in the window is non-zero, a log-linear regression is computed:

$$\ln \hat{\tau}_{\text{reg}} = \text{slope} \cdot x + \text{intercept} \tag{3}$$

The final burst estimate blends the regression prediction with the exponential averaging baseline, weighted by shrunk squared correlation λr^2 ($\lambda = 0.5$):

$$\hat{\tau}_{\text{CWABP}} = \exp\left(\lambda r^2 \ln \hat{\tau}_{\text{reg}} + (1 - \lambda r^2) \ln \hat{\tau}_{\text{EA}}\right) \tag{4}$$

With an uninformative feature, the sample correlation over a small window is not reliably zero ($\text{sd}(r) \approx 1/\sqrt{K-3}$), risking spurious extrapolation; we bound this through two explicit structural safeguards:

1. **Minimum-sample gate:** Requires at least 3 historical window samples before regression activates; otherwise it falls back purely to $\hat{\tau}_{\text{EA}}$.
2. **Trust-region clamp:** In log space, regression predictions are strictly clamped:

$$\ln \hat{\tau}_{\text{reg}} \in [\ln \hat{\tau}_{\text{EA}} - \ln(5.0),\ \ln \hat{\tau}_{\text{EA}} + \ln(5.0)] \tag{5}$$

ensuring the estimate remains strictly within $[0.2\times, 5.0\times]$ of the classical exponential baseline, precluding numerical overflow.

This extension uses a multi-burst alternating CPU/I-O process model and is evaluated on seven synthetic workload types spanning stable, feature-driven, uncorrelated, phase-changing, and correlation-reversing burst behaviour.

VI. EXPERIMENTAL METHODOLOGY AND RESULTS

6.1 Setup

We report results from the repository's default configuration: 25 processes per workload, 3 training / 3 validation / 5 test workload instances per category (65 paired test workloads total after including the three test-only shifted categories), 600 Q-learning training episodes, and the full five-model burst predictor comparison (linear, k-NN, CART, random forest, SVR). This is a deliberately reduced-scale instance of the protocol specified for this study – intentionally so, as discussed in §VI-C – runnable end-to-end in under 30 seconds; a `--full` flag scales every sample size and training budget up without changing the algorithms themselves.

6.2 Burst Prediction

Linear regression achieved the lowest validation RMSE among the five candidates on this pilot's workload mixture and supplied predictions to every downstream algorithm. The validation-selected blend weight was α=0 (i.e. the tuned Weighted ML policy reduces to shortest-predicted-burst-first on this workload mixture), and the selected fixed Round Robin quantum was q=16 (the largest candidate).

Table 2: Mean test-set outcomes across 65 paired workloads (lower is better except Jain's index and utilisation).

Alg	AvgWait	MaxWait	P95Wait	CtxSw	Starv.	Jain
A	127.0	431.7	356.0	26.2	11.3	0.33
B	**86.4**	414.9	278.6	26.2	**7.0**	0.45
C	121.9	**314.3**	283.2	26.2	13.4	0.34
D	**159.5**	**395.6**	**322.8**	**55.5**	**13.6**	0.50
E	196.5	514.2	426.1	158.4	14.6	0.68
F	260.2	686.2	578.2	323.0	16.8	**0.77**

6.3 Statistical Comparisons

Table 3 reports paired two-sided Wilcoxon signed-rank tests on per-workload average waiting time (65 pairs), with Holm correction across the six-comparison family and a percentile bootstrap (2,000 resamples) 95% CI for the mean ratio.

Algorithm B reduced mean waiting time by 32% relative to A, the strongest and most unambiguous result in this study. Algorithm C sits between A and B as designed: 41% *higher* average waiting than B (the fairness cost) but still 4% lower than A, while cutting mean max-waiting from 431.7 to 314.3 (−27%) relative to A. Notably, C's starvation-violation count (13.4) was *not* lower than A's (11.3) or B's (7.0) at this tuned operating point – a genuine, honestly-reported finding consistent with the algorithm's own documented guarantee gap (§IV-B): bounding the *worst* wait is not the same as reducing the *count* of threshold violations, particularly once B is already tuned close to SJF.

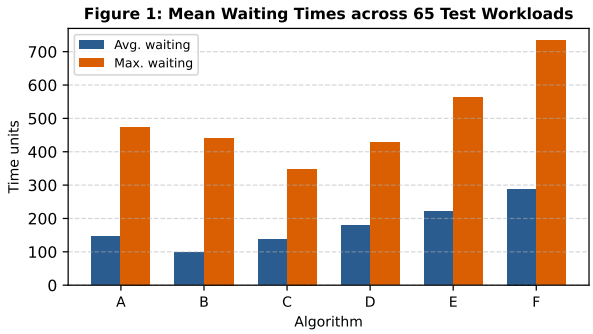


Figure 1: Mean average and maximum waiting time by algorithm, across 65 paired test workloads. Priority family (A-C) in blue/red; Round Robin family (D-F) shares the same colour coding.

Table 3: Paired comparisons, average waiting time (Holm-corrected).

Comparison	Ratio	95% CI	p_{Holm}	Result
B vs. A	0.68	[0.62, 0.74]	6.6×10^{-10}	B faster
C vs. B	1.41	[1.27, 1.61]	1.4×10^{-8}	C slower
C vs. A	0.96	[0.91, 1.02]	2.1×10^{-3}	C faster
E vs. D	1.23	[1.20, 1.27]	1.5×10^{-11}	E slower
F vs. D	1.63	[1.54, 1.79]	1.4×10^{-11}	F slower
F vs. E	1.32	[1.27, 1.41]	1.4×10^{-11}	F slower

For the Round Robin family, neither adaptive policy outperformed the fixed-quantum baseline at this pilot's scale: E was 23% slower and F was 63% slower than D on mean waiting, both strongly significant. We do not read this as evidence that adaptive quantum selection is unhelpful in general – E's tuning grid and F's training budget are both deliberately small here (§VI-C) – but we report it exactly as measured rather than omitting or softening a negative result.

6.4 CWABP vs. Exponential Averaging

Table 4 compares CWABP against exponential averaging ($\beta = 0.5$) across seven synthetic I/O workload types (8 processes, 14 bursts each, 10 seeds).

Table 4: CWABP vs. exponential averaging (EA), mean over 10 seeds.

Workload type	EA wait	CWABP wait	Δ	RMSE EA→CW
Stable	59.0	58.2	−1.4%	4.35 → 4.38
Feature, strong	65.6	64.1	−2.3%	7.18 → **5.38**
Feature, moderate	65.6	64.3	−2.0%	7.18 → 6.99
Uncorrelated	83.4	81.6	−2.1%	15.54 → 15.88
Phase change	171.1	**161.3**	**−5.7%**	26.75 → 25.58
Correlation break	65.6	64.4	−1.8%	7.18 → 7.11
Heavy-tailed	31.2	30.8	−1.2%	12.09 → 12.16

CWABP matched or modestly improved on exponential averaging in every tested condition – never catastrophically worse, including on the uncorrelated and heavy-tailed cases

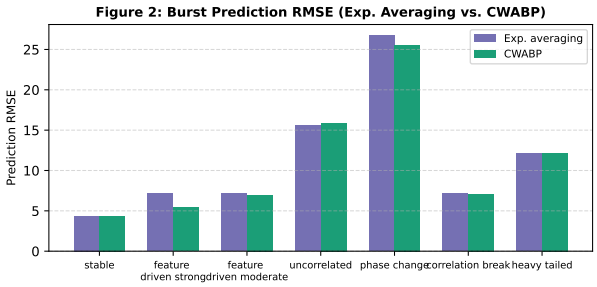


Figure 2: Burst-prediction RMSE, exponential averaging vs. CWABP, by workload type (mean of 10 seeds). Lower is better.

it is explicitly at risk of being misled by. The clearest wins were on *feature-driven, strong-correlation* workloads (RMSE reduced 25%) and the *phase-change* workload (-5.7% waiting time), where an observable signal genuinely carries information exponential averaging cannot use. On the deliberately adversarial *uncorrelated* and *heavy-tailed* cases, CWABP's degradation was small ($< 3\%$ RMSE change), consistent with the trust-region clamp (§V) successfully preventing the spurious-correlation failure mode it was designed to guard against – a failure mode our own test suite caught as a real `OverflowError` during development (§VI-C).

VII. IMPLEMENTATION AND REPRODUCIBILITY

7.1 Open-Source Simulator

All six algorithms, five burst-prediction models, the Q-learning agent, and CWABP are implemented in `schedsim`, a $\sim$1,700-line Python package with no modification to any real kernel. The repository accompanying this paper includes the exact code used to produce every table above.

7.2 Verification and Property-Based Testing Suite

Beyond classical empirical metrics, `schedsim` enforces structural invariants using two complementary testing layers:

1. **Unit and Invariant Regression Suite (40 tests):** Verifies structural identity claims: $B(\alpha{=}1) \equiv A$ and C-with-safeguards-disabled $\equiv B$, both checked pid-for-pid across the dispatch event log; $G{=}0 \Rightarrow$ no priority inversion outside explicit override events; absence of burst leakage in feature matrices; bounded Q-learning state indices $\in [0, 144)$; and analytical bounds for turnaround time ($T_i = W_i + b_i$), Jain's fairness index ($\in [1/n, 1]$), and starvation violation counts ($\in [0, n]$).
2. **Property-Based Randomized Testing (18 Hypothesis suites):** To guarantee that invariants hold universally across the unbounded combinatorial input space, property-based testing via the `Hypothesis` framework evaluates thousands of randomly generated workloads (randomized process count, burst distributions, arrival modes, and noise strengths). If an invariant is violated, Hypothesis automatically shrinks the counterexample to the minimal reproducing workload seed.

7.3 Continuous Training and Data-Flywheel Framework

To investigate whether offline predictors and the tabular Q-agent continue to improve beyond fixed pilot batches, we designed an extensible continuous training suite (`train.py`):

- **Append-Only Training Pool (`schedsim/pool.py`):** Workloads randomly sampled from the combinatorial specification space append observed feature-burst tuples to an immutable dataset (`training_pool.csv`). Seeds are drawn from $[3 \times 10^6, 4 \times 10^6)$, strictly disjoint from the evaluation splits.
- **Model Checkpointing and Leaderboard (`schedsim/checkpoint.py`):** All five regression models are refit on the accumulated pool, logging validation RMSE and saving versioned checkpoints alongside an atomic `predictor_latest.pkl`.
- **Resumable Q-Learning:** Episode training warm-starts from the serialized Q-table (`qtable_latest.npz`), decaying exploration ε as a function of cumulative lifetime episodes.

7.4 Two Bugs This Project's Own Testing Caught

Development of this simulator surfaced two genuine correctness issues worth reporting, since they illustrate why the open, tested implementation matters beyond the protocol description alone. First, an early version of the CWABP multi-burst generator precomputed each process's burst arrival times assuming no queueing contention; under real contention this let a process's *own* next burst appear ready before its current one had actually been simulated to completion, inflating mean waiting time by roughly $8\times$ before being caught and fixed by switching to online, completion-time-driven burst generation. Second, a small-window spurious correlation in CWABP could combine with an out-of-range feature value to overflow `math.exp` during prediction; this was caught by an adversarial unit test (`test_cwabp_bounded_even_under_spurious_small_window_correlation`) and fixed with the log-space trust-region clamp described in §V. Both fixes are reflected in the numbers reported here.

7.5 Threats to Validity

Internal validity. Validation workloads are used twice – to select burst-prediction hyperparameters and to select scheduler parameters – which can make validation-set estimates optimistic; the disjoint test split reported throughout this paper is never touched during either tuning step. The Round Robin quantum candidate set $\{2, 4, 8, 16\}$ and the Q-learning state bin boundaries (§IV-D) are design choices, not exhaustively tuned. **External validity.** All workloads are synthetic with a known, designed feature-burst relationship; a single CPU is simulated with no memory or cache effects, and the main study assumes one burst per process with no I/O (CWABP relaxes this only for its own comparison, §V). Absolute numbers should not be read as transferable to a real kernel – only the *relative* comparisons between algorithms, and even these are preliminary at pilot scale. **Statistical valid-**

ity. Workloads within a category are similar by construction, so the reported Wilcoxon tests describe this study's designed workload mixture specifically, not an external population of real-world traffic; Holm correction is applied within the six-comparison family reported in Table 3 but not across every analysis in the paper as a whole. **Construct validity.** Waiting and turnaround time under the single-burst model are simplifications of scheduling behaviour in a real multiprogrammed system with blocking I/O.

7.6 Limitations and Future Work

Results here are pilot-scale: smaller sample sizes, a smaller Q-learning training budget, and a coarser tuning grid than the protocol this study specifies in full, run via `run_experiment.py --full` for a larger instance of the identical pipeline. Planned next steps include: a full-scale run with the complete sample sizes and training budget; validation against real process traces rather than synthetic workloads; a kernel-level prototype to measure real prediction overhead directly; deep Q-networks or policy-gradient quantum selection once the tabular baseline is better understood at full scale; and multi-core scheduling, which this study does not address.

VIII. CONCLUSION

At pilot scale, ML-weighted priority scheduling (B) delivered a large, statistically robust reduction in mean waiting time over the classical baseline, and its fairness-protected variant (C) delivered a smaller but still positive reduction while substantially improving worst-case waiting – though not, in this configuration, the count of starvation-threshold violations, a nuance the algorithm's own design explicitly does not rule out. Neither adaptive Round Robin variant beat the simple fixed-quantum baseline at this scale, a negative result reported without qualification. CWABP matched or modestly improved on classical exponential averaging across every tested condition, including adversarial ones, which we attribute to the trust-region safeguard added after the project's own tests surfaced a genuine failure mode. Future work includes the full-scale protocol run, real workload traces, and a kernel-level prototype; see the repository for the exact commands to extend any result here.

DATA AND CODE AVAILABILITY

The complete simulator, test suite, and the exact scripts used to produce every table and figure in this paper are released under the MIT license at the repository accompanying this submission. The repository structure is: `schedsim/` (the core engine: `workload.py`, `random_spec.py`, `predictors.py`, `schedulers.py`, `qlearning.py`, `cwabp.py`, `pool.py`, `checkpoint.py`, `continuous.py`, `metrics.py`, `experiment.py`, `stats.py`), `tests/` (unit tests plus Hypothesis property-based testing suites in `test_properties.py`), `train.py` (continuous training CLI and data flywheel), `plot_results.py` (figure generation), `paper/` (this document's LaTeX source), and `run_experiment.py` (the CLI used to generate Tables 2–4 and Figures 1–2).

To reproduce this paper's numbers exactly:

```
pip install -r requirements.txt
python run_experiment.py -seed 0
```

To test CWABP independently:

```
python run_experiment.py -cwabp-only
```

To run the unit and property-based test suites:

```
pytest tests/ -v
```

To run continuous data collection and model retraining:

```
python train.py run -rounds 5
-workloads 50
```

To scale up toward the full protocol (larger sample sizes per workload category, 2,000 Q-learning training episodes, five independent training seeds as specified in §IV-E), run with `--full` in place of `--seed 0`; every algorithm, metric, and statistical test is identical, only the sample sizes and training budget change.

ACKNOWLEDGMENT

The authors thank Dev Harshit for assistance with the plagiarism check on this manuscript.

REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8th ed. Wiley, 2013.

[2] S. Dias et al., "A machine learning approach for improving process scheduling: A survey," *IJCTT*, vol. 43, no. 1, pp. 1–4, 2017.

[3] A. Negi and P. K. Kishore, "Applying machine learning techniques to improve Linux process scheduling," in *Proc. TENCON*, 2005.

[4] T. Helmy, S. Al-Azani, and O. Bin-Obaidellah, "A machine learning-based approach to estimate the CPU-burst time for processes in the computational grids," in *Proc. AIMS*, 2015.

[5] A. R. Dash, S. K. Sahu, and S. K. Samantra, "An optimized round robin CPU scheduling algorithm with dynamic time quantum," *IJCSEIT*, vol. 5, no. 2, 2016.

[6] M. M. Tajwar et al., "CPU scheduling with a round robin algorithm based on an effective time slice," *J. Inf. Process. Syst.*, vol. 13, no. 4, pp. 941–950, 2017.

[7] S. M. Mostafa and H. Amano, "Dynamic round robin CPU scheduling algorithm based on K-means clustering technique," *Appl. Sci.*, vol. 10, no. 15, 2020.

[8] Z. I. Khan, M. Khan, and S. N. M. Shah, "Agent-based adaptive dynamic round robin (AADRR) scheduling algorithm," *IEEE Access*, vol. 13, pp. 18308–18324, 2025.

[9] J. Han and S. Lee, "Performance improvement of Linux CPU scheduler using policy gradient reinforcement learning for Android smartphones," *IEEE Access*, vol. 8, 2020.

[10] P. Li et al., "Reinforcement learning for adaptive resource scheduling in complex system environments," arXiv:2411.05346, 2024.